

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И
ИНФОРМАТИКИ

Кафедра «Программная инженерия»

Лабораторная работа № 6
по дисциплине «Введение в информационные технологии»
по теме: «Основы работы с библиотекой NumPy»

Выполнил студент группы:

БВТ2504

Гапеев Алексей Денисович

Проверил: Харрасов К.Р.

Москва 2026

Оглавление

Цель работы.....	3
Входные данные.....	4
Задание 1.....	4
Задание 2.....	6
Задание 3.....	9
Задание 4.....	11
Задание 5.....	12
Задание 6.....	13
Задание 7.....	14
Вывод.....	16

Цель работы:

Изучить основные возможности библиотеки NumPy для работы с многомерными массивами, освоить встроенные функции обработки данных, понять механизм векторизации и broadcasting, а также реализовать базовые операции над массивами без использования NumPy с целью сравнения удобства и производительности.

Входные данные

Массивы, используемые в лабораторной работе (файл arrays.py):

```
a = np.array([4, 5, 2, 2])
b = np.array([[12, 34, 2, 6],[3.2, 4.5, 63.5, 74.8]])
c = [15, 64, 1, 30]
d = [[12, 34, 2, 6],
     [3, 4, 63, 74]]

e = np.array([[43, 6, 334, 234],[645, 12315, 6, 457]])
f = np.array([234, 5, 14, 4])

int_a = np.array([4, 5, 2, 2], dtype=np.int32)
float_a = np.array([4, 5, 2, 2], dtype=np.float64)
```

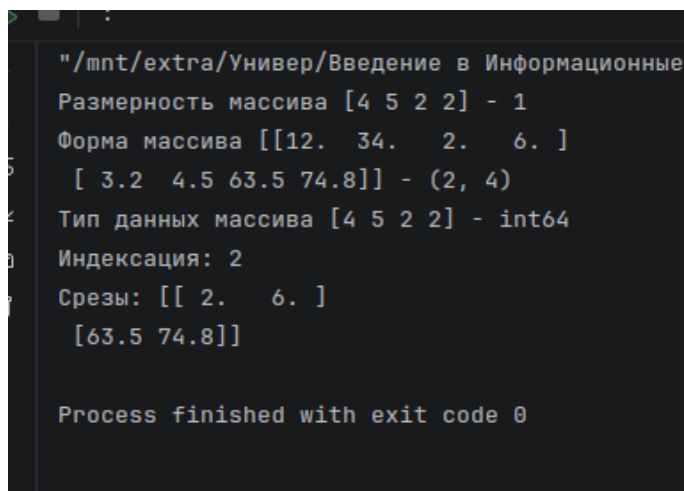
Задание 1

Создайте одномерный и двумерный массивы NumPy, содержащие числовые данные. Для каждого массива определите размерность, форму, тип данных и количество элементов. Продемонстрируйте доступ к элементам массива с использованием индексации и срезов. Результаты выполнения операций зафиксируйте и прокомментируйте.

```
print(f"Размерность массива {arrays.a} - {arrays.a.ndim}\nФорма массива {arrays.b} - {arrays.b.shape}\nТип данных массива {arrays.a} - {arrays.a.dtype}")

# Доступ к элементам массива

print(f"Индексация: {arrays.a[2]}\nСрезы: {arrays.b[:2, 2:]}")
```



```
"/mnt/extra/Универ/Введение в Информационные
Размерность массива [4 5 2 2] - 1
Форма массива [[12.  34.   2.   6. ]
 [ 3.2  4.5 63.5 74.8]] - (2, 4)
Тип данных массива [4 5 2 2] - int64
Индексация: 2
Срезы: [[ 2.   6. ]
 [63.5 74.8]]

Process finished with exit code 0
```

Фигура 1: Результат выполнения кода для задания 1

Массив А одномерный, количество элементов 11, все элементы целочисленные.

При индексации выводится элемент массива А с индексом 3, при срезе выводятся элементы с 2 по 6 индекс

Задание 2

Для созданных массивов выполните вычисление суммы, среднего значения, минимального и максимального элементов, а также стандартного отклонения с использованием встроенных функций NumPy. Продемонстрируйте поэлементные арифметические операции между массивами, а также между массивом и скаляром. Дополнительно реализуйте аналогичные операции с использованием стандартных средств Python без применения библиотеки NumPy. Сравните полученные результаты и сделайте вывод о различиях в удобстве и выразительности кода.

```
print(f"\nСумма: {arrays.a + 12.5}\nРазность: {arrays.a - 10}\nСреднее значение: {arrays.a.mean()}\nМинимальное: {arrays.a.min()}\n"
      f"Максимальное: {arrays.a.max()}\nСреднее отклонение: {arrays.a.std()} ")
```

```
print(f"\nОперации с массивами")
print(f"Сумма: {arrays.a + arrays.b}, Произведение: {arrays.a * arrays.b}")
# Сумма
```

```
sum_number = 5
new_array = []
for i in range(len(arrays.c)):
    new_array = arrays.c[i] + 12.5
print(f"Сумма: {new_array}")

# Среднее значение
average = 0
for i in arrays.c:
    average += i
print(f"Среднее значение: {average/(len(arrays.c))}")
```

```
# Минимальное значение
```

```
minimal = arrays.c[0]

for i in arrays.c:
    if i < minimal:
        minimal = i
    else: continue

print(f"Минимальное значение: {minimal}")
```

```
# Максимальное значение
```

```
maximal = arrays.c[0]
```


Вывод: Код, сделанный с помощью библиотеки NumPy получился более структурированным и компактным, в отличие от реализации тех же функций с помощью стандартных библиотек Python.

Задание 3

Исследуйте влияние формы массива на выполнение вычислений. Выполните агрегатные операции по строкам и столбцам двумерного массива с использованием параметра `axis`. Реализуйте аналогичные вычисления без использования NumPy, применяя циклы и стандартные структуры данных Python, и сравните подходы.

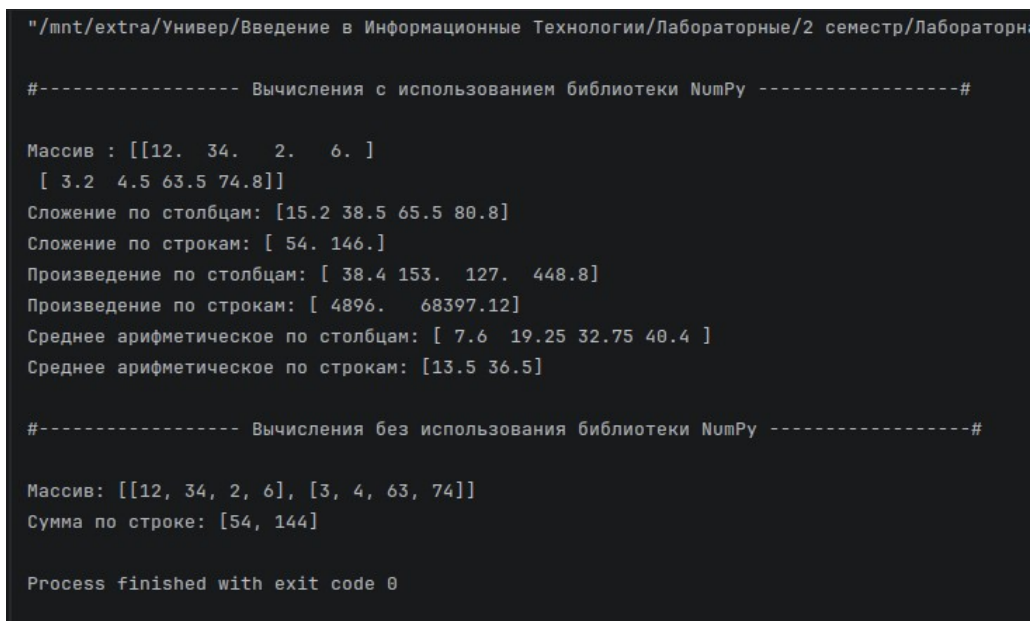
```
print(f"Массив : {arrays.b}")

print(f"Сложение по столбцам: {np.sum(arrays.b, axis=0)}")
print(f"Сложение по строкам: {np.sum(arrays.b, axis=1)}")
print(f"Произведение по столбцам: {np.prod(arrays.b, axis=0)}")
print(f"Произведение по строкам: {np.prod(arrays.b, axis=1)}")
print(f"Среднее арифметическое по столбцам: {np.mean(arrays.b, axis=0)}")
print(f"Среднее арифметическое по строкам: {np.mean(arrays.b, axis=1)}")

print(f"Массив: {arrays.d}")

# Сложение по строке

summary_array = []
for i in range(len(arrays.d[:])):
    summary = 0
    for j in range(len(arrays.d[0][:])):
        # print(f"i = {i}, j = {j}")
        summary = summary + arrays.d[i][j]
    summary_array.append(summary)
print(f"Сумма по строке: {summary_array}")
```



```
"/mnt/extra/Универ/Введение в Информационные Технологии/Лабораторные/2 семестр/Лабораторн

#----- Вычисления с использованием библиотеки NumPy -----#

Массив : [[12.  34.   2.   6. ]
 [ 3.2  4.5 63.5 74.8]]
Сложение по столбцам: [15.2 38.5 65.5 80.8]
Сложение по строкам: [ 54. 146.]
Произведение по столбцам: [ 38.4 153. 127. 448.8]
Произведение по строкам: [ 4896.   68397.12]
Среднее арифметическое по столбцам: [ 7.6 19.25 32.75 40.4 ]
Среднее арифметическое по строкам: [13.5 36.5]

#----- Вычисления без использования библиотеки NumPy -----#

Массив: [[12, 34, 2, 6], [3, 4, 63, 74]]
Сумма по строке: [54, 144]

Process finished with exit code 0
```

Фигура 3: Результат выполнения кода для задания 3

Вывод: код, в котором используются функции NumPy, более компактный, в нем не нужно вводить промежуточные переменные и создавать циклы, что занимает больше ресурсов ПК.

Задание 4

Продемонстрируйте механизм broadcasting при выполнении арифметических операций между массивами различной формы, а также между массивом и скаляром. Объясните, почему реализация аналогичных операций без NumPy требует использования вложенных циклов и приводит к усложнению кода.

```
print(f"Массивы:\nf: {arrays.f}\ne: {arrays.e}")
broadcasting_summary = arrays.f + arrays.e
broadcasting_submit = arrays.e - arrays.f
print(f"Сумма: {broadcasting_summary}\nРазность: {broadcasting_submit}")
```

```
"/mnt/extra/Универ/Введение в Информационные Технологии/Лабораторные/2 семестр
#----- Использование метода broadcasting -----#

Массивы:
f: [234  5 14  4]
e: [[ 43  6 334 234]
 [ 645 12315 6 457]]
Сумма: [[ 277 11 348 238]
 [ 879 12320 20 461]]
Разность: [[ -191 1 320 230]
 [ 411 12310 -8 453]]

Process finished with exit code 0
|
```

Фигура 4: Результат выполнения кода для задания 4

Вывод: Для повторения подобного механизма подстройки массивов для дальнейшего произведения вычислений необходимо сначала узнать, какой из массивов больше и на сколько, расширить остальные массивы недостающими элементами, а потом проводить вычисления с помощью циклов

Задание 5

Создайте логические маски для массива NumPy и выполните фильтрацию элементов по заданному условию. Реализуйте аналогичную фильтрацию без использования NumPy.

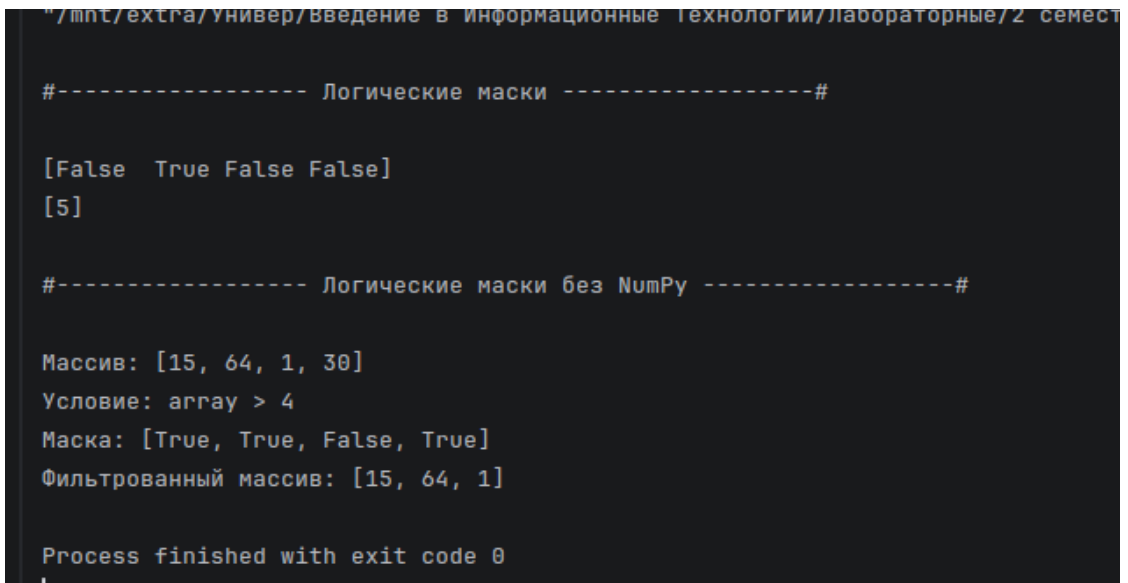
```
mask = arrays.a > 4
print(mask)
print(arrays.a[mask])

mask = []

for i in arrays.c:
    if i < 4:
        mask.append(False)
    else: mask.append(True)

# Без использования NumPy
# print(mask)
filtered_massive = []
count = 0
for j in mask:
    if j:
        filtered_massive.append(arrays.c[count])
    else: continue
    count += 1

print(f"Массив: {arrays.c}\nУсловие: array > 4\nМаска: {mask}\nФильтрованный массив: {filtered_massive}")
```



```
"/mnt/extra/универ/введение в информационные технологии/лабораторные/2 семестр

#----- Логические маски -----#

[False  True False False]
[5]

#----- Логические маски без NumPy -----#

Массив: [15, 64, 1, 30]
Условие: array > 4
Маска: [True, True, False, True]
Фильтрованный массив: [15, 64, 1]

Process finished with exit code 0
```

Фигура 5: Результат выполнения кода для задания 5

Задание 6

Выполните сортировку массива и определите индекс максимального элемента с использованием функций NumPy. Реализуйте аналогичные операции с применением стандартных средств Python.

```
sorted_massive_a = np.sort(arrays.a)
max_sorted_massive = np.argmax(sorted_massive_a)
print(sorted_massive_a)
print(f"Максимальное значение массива {arrays.a}: {sorted_massive_a[max_sorted_massive]}")
```

```
# Без NumPy
```

```
max_value = arrays.c[0]
```

```
for i in arrays.c:
    if max_value <= i:
        max_value = i
```

```
"/mnt/extra/универ/Введение в Информационные Технологии/лабораторные/2 семестр/лабораторная 6/.venv/bin/python3
#----- Сортировка массива и нахождение максимального значения -----#

[2 2 4 5]
Максимальное значение массива [4 5 2 2]: 5

#----- Сортировка массива и нахождение максимального значения без NumPy -----#

Максимальное значение массива [15, 64, 1, 30]: 64

Process finished with exit code 0
```

Фигура 6: Результат выполнения кода для задания 6

Задание 7

Измените тип данных массива NumPy и проанализируйте влияние типа данных на результаты вычислений и объём используемой памяти. Сгенерируйте массив большого размера и сравните время выполнения операции суммирования с использованием NumPy и стандартных средств Python. Сделайте вывод о различиях в производительности.

```
print(f"\n#----- Количество занимаемой памяти -----#\n")

print(f"Тип данных int: {arrays.int_a.nbytes}")
print(f"Тип данных float: {arrays.float_a.nbytes}")

print(f"\n#----- Скорость выполнения инструкций -----#\n")

a = 20000000

python_list = list(range(a))

numpy_array = np.arange(a)

start = time.time()
sum(python_list)
time_python = time.time() - start

start = time.time()
np.sum(numpy_array)
time_numpy = time.time() - start

print(f"Время выполнения инструкций с обычным массивом: {time_python}\nВремя выполнения инструкций с NumPy массивом: {time_numpy}")
print(f"Разница в скорости выполнения: {time_python // time_numpy}")
```

```
#----- Количество занимаемой памяти -----#

Тип данных int: 16
Тип данных float: 32

#----- Скорость выполнения инструкций -----#

Время выполнения инструкций с обычным массивом: 0.0647590160369873
Время выполнения инструкций с NumPy массивом: 0.006673097610473633
Разница в скорости выполнения: 9.0

Process finished with exit code 0
|
```

Фигура 7: Результат выполнения кода для задания 7

Вывод: разница между используемыми типами данных заключается в количестве используемой памяти, которая выделяется для хранения каждой переменной. Время выполнения операций с обычными массивами и массивами NumPy отличается в разы, что показывает, что для работы с большими данными лучше использовать библиотеку NumPy.

Вывод

В результате выполнения лабораторной работы была изучена библиотека NumPy, операции с массивами NumPy, а также проведено сравнение в производительности и занимаемой памяти кодом, написанным с использованием библиотеки NumPy и без нее